ใบแล็บ (เอกสารสเปคอ้างอิง)

การออกแบบวงจร AES-128 Encryption ด้วย Generative AI

รายวิชา Digital System Design with Verilog — บทที่ 9: การใช้ AI ช่วยออกแบบวงจรดิจิทัล

> **เอกสารนี้คือสเปค ไม่ใช่คำตอบสำเร็จรูป**
>
> เนื้อหาทั้งหมดในเอกสารนี้เป็นข้อกำหนด (spec) ที่นักศึกษาต้องนำไปใช้เป็นข้อมูลตั้งต้นป้อนให้ generative AI ช่วยออกแบบวงจร — ไม่ใช่ไฟล์ RTL สำเร็จรูป นักศึกษาต้องออกแบบ/สร้าง/ตรวจสอบวงจรเองด้วยความช่วยเหลือของ AI ตาม workflow ที่เรียนในบทที่ 9 ตัวเลขตัวอย่าง input/output ทุกจุดในเอกสารนี้คำนวณจริงและตรวจสอบถูกต้องแล้ว (อ้างอิงค่ามาตรฐาน NIST FIPS-197) ใช้เป็นเกณฑ์ตรวจสอบผลลัพธ์ของวงจรที่สร้างได้ทันที

0. คำชี้แจงและวัตถุประสงค์ของแล็บ

แล็บนี้มีโจทย์เดียว: ใช้ generative AI (เช่น Claude, ChatGPT, Gemini หรือเครื่องมือ AI coding assistant อื่น) ช่วยออกแบบและสร้างวงจร AES-128 Encryption (เข้ารหัสอย่างเดียว ไม่ต้องทำวงจรถอดรหัส) เป็นภาษา Verilog โดยเดินตาม AI-Assisted Workflow ที่เรียนในหัวข้อ 4 ของบทที่ 9 อย่างครบถ้วน:

1. 4.1 เขียนสเปคให้ AI เข้าใจ — ใช้เนื้อหาในเอกสารนี้ (หัวข้อ 1–7) เป็นข้อมูลตั้งต้น
2. 4.2 ให้ AI generate โค้ด RTL ทีละ submodule ตามสถาปัตยกรรมในหัวข้อ 6
3. 4.3 Code review ร่วมกับ AI — อ่านโค้ดทุกบรรทัดเทียบกับสเปค
4. 4.4 ให้ AI สร้าง testbench (unit test ทีละโมดูลแบบ bottom-up + integration test รวมระบบ)
5. 4.5 รัน simulation จริง (Icarus Verilog หรือ ModelSim) — ห้ามเชื่อผลจาก AI เฉย ๆ โดยไม่รันจริง
6. 4.6 Debug loop กับ AI จนผ่านครบทุกเทส

สิ่งที่ต้องส่งมอบระบุไว้ในหัวข้อ 9 ท้ายเอกสาร

1. AES-128 คืออะไร และทำไมต้องมีวงจรนี้

สถานการณ์

ข้อมูลที่ส่งผ่านเครือข่ายหรือจัดเก็บในอุปกรณ์ ต้องถูกเข้ารหัสก่อนแทบทุกครั้งในระบบสมัยใหม่ — การเชื่อมต่อเว็บแบบ HTTPS/TLS, เครือข่ายไร้สาย WiFi (WPA2/WPA3), การเข้ารหัสฮาร์ดดิสก์/SSD ทั้งลูก (full-disk encryption), และการอัปเดตเฟิร์มแวร์อุปกรณ์ IoT อย่างปลอดภัย ล้วนใช้อัลกอริทึม AES (Advanced Encryption Standard) เป็นแกนกลางของการเข้ารหัส

ปัญหาที่ต้องแก้

การเข้ารหัสด้วยซอฟต์แวร์ (รันบน CPU) เพียงพอสำหรับงานทั่วไป แต่ไม่เพียงพอสำหรับงานที่ต้องการ throughput สูงมาก เช่น สายสัญญาณความเร็วสูง หรือ SSD ที่ต้องเข้ารหัสทุกไบต์ที่เขียนแบบ real-time หรือระบบฝังตัวที่ CPU มีสมรรถนะจำกัดและต้องประหยัดพลังงาน — งานเหล่านี้ต้องการวงจรฮาร์ดแวร์เฉพาะทาง (AES hardware accelerator หรือ crypto IP core) ที่คำนวณเร็วกว่าและประหยัดพลังงานกว่าการรันด้วยซอฟต์แวร์มาก

หน้าที่ของวงจรนี้

รับ plaintext ขนาด 128 บิต (16 ไบต์) และ cipher key ขนาด 128 บิต เข้าวงจร แล้วคำนวณ ciphertext ขนาด 128 บิตออกมาตามอัลกอริทึม AES-128 มาตรฐานที่กำหนดโดย NIST FIPS-197 ให้ถูกต้องทุกบิต ภายในเวลาจำกัด (ระดับหลักสิบ clock cycle) — วงจรนี้ทำหน้าที่ "เข้ารหัส" (encrypt) เท่านั้น ไม่ต้องทำวงจรถอดรหัส (decrypt)

2. สเปคระบบ (Top-Down): Input / Output ของวงจร AES-128 Top Module

อินเทอร์เฟซระดับบนสุดที่ AI ต้องรู้ก่อนเริ่มแตกเป็น submodule:

สัญญาณ	ทิศทาง	บิต	ความหมาย
`clk`	in	1	สัญญาณนาฬิกาหลัก
`rst_n`	in	1	รีเซ็ต active-low (reset ทุก state ภายในให้ค่าเริ่มต้น)
`key_in[127:0]`	in	128	cipher key ที่จะใช้เข้ารหัส
`key_valid`	in	1	pulse 1 cycle: key_in พร้อมให้โหลด → เริ่ม key expansion
`data_in[127:0]`	in	128	plaintext block ที่จะเข้ารหัส
`start`	in	1	pulse 1 cycle: data_in พร้อม และ key expansion เสร็จแล้ว → เริ่มเข้ารหัส
`busy`	out	1	1 = กำลังทำ key expansion หรือกำลังเข้ารหัสอยู่
`data_out[127:0]`	out	128	ciphertext ที่ได้ — valid เมื่อ done=1
`done`	out	1	pulse 1 cycle: data_out พร้อมใช้งานแล้ว

3. กติกาและ Edge Case ที่ต้องจัดการ

7. ต้อง key_valid อย่างน้อย 1 ครั้งก่อน start ครั้งแรกเสมอ — ถ้า start มาขณะ key expansion ยังไม่เสร็จ (busy=1) ให้วงจร "เพิกเฉย" (ignore) พัลส์ start นั้น ไม่ใช่ค้างหรือ error
8. เมื่อ key expansion เสร็จแล้ว (busy กลับเป็น 0) key เดิมถูกเก็บไว้ใช้เข้ารหัสได้หลาย plaintext ติดกัน โดยไม่ต้อง key_valid ซ้ำ — ต้อง key_valid ใหม่เฉพาะตอนเปลี่ยน key เท่านั้น
9. ระหว่าง busy=1 ถ้ามี start หรือ key_valid ใหม่เข้ามาซ้ำ ให้เพิกเฉยจนกว่า busy จะกลับเป็น 0
10. Round ที่ 10 (รอบสุดท้าย) ห้ามมี MixColumns เด็ดขาด — เป็นกติกาตายตัวของมาตรฐาน AES-128 ไม่ใช่ตัวเลือกในการออกแบบ
11. data_out ไม่จำเป็นต้อง reset ค่าระหว่างรอบ — ค่าที่ค้างอยู่คือ ciphertext ล่าสุดที่เคย done จนกว่าจะมี done ใหม่มาทับ

4. พฤติกรรมวงจรโดยรวม (Overall Behavior)

สถาปัตยกรรมที่แนะนำ

แนะนำให้ออกแบบเป็นแบบ "iterative" — มี datapath สำหรับคำนวณ 1 round เพียงชุดเดียว แล้วใช้ FSM ควบคุมให้หมุนใช้ซ้ำ 10 รอบ (ประหยัดพื้นที่ฮาร์ดแวร์กว่าการ unroll ทั้ง 10 round เป็นวงจร pipeline ขนานทั้งหมด) — นี่เป็นข้อแนะนำ ไม่ใช่ข้อบังคับตายตัว นักศึกษาสามารถออกแบบสถาปัตยกรรมภายในเองได้ ตราบใดที่ผลลัพธ์ output ตรงตามลำดับการคำนวณที่กำหนดด้านล่าง

ลำดับการคำนวณ (ตายตัวตามมาตรฐาน AES-128 — ห้ามเปลี่ยน)

Round 0: `state = plaintext XOR round_key[0]` (AddRoundKey เท่านั้น)

Round 1..9: `state = AddRoundKey( MixColumns( ShiftRows( SubBytes(state) ) ), round_key[r] )`

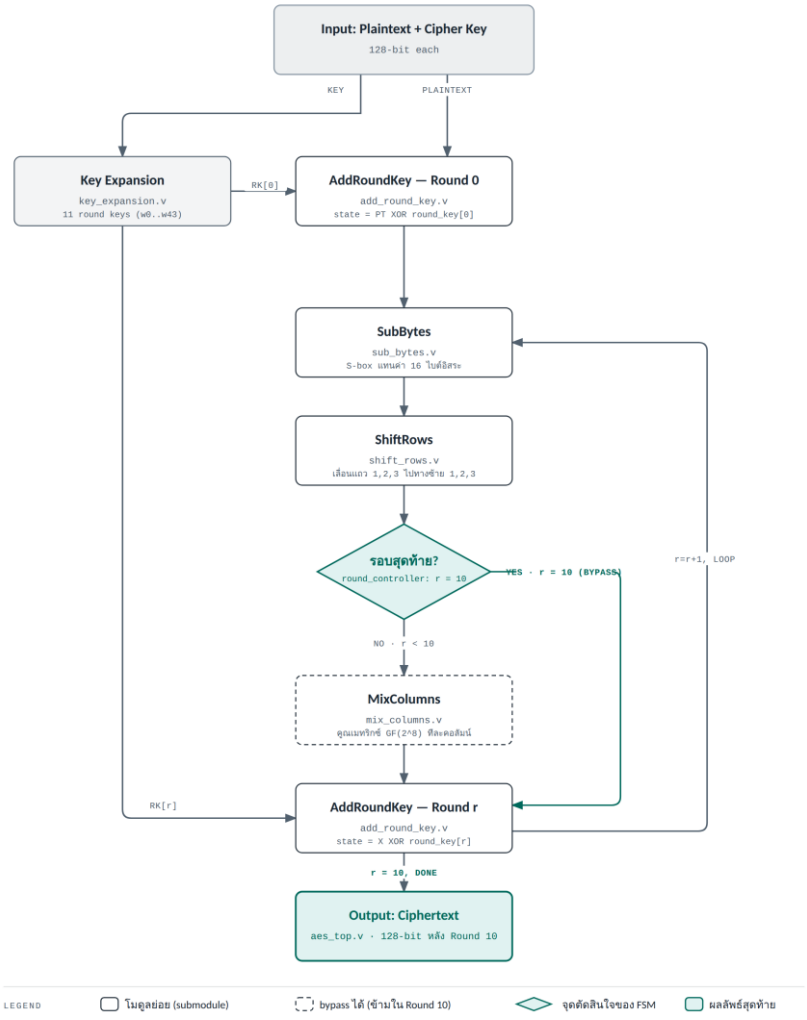
Round 10: `state = AddRoundKey( ShiftRows( SubBytes(state) ), round_key[10] )` ← ไม่มี MixColumns

`ciphertext = state` หลัง Round 10

Key Expansion ทำงานคู่ขนาน/ก่อนหน้าการเข้ารหัส: จาก cipher key 128 บิต จะขยายเป็น round key จำนวน 11 ชุด (round_key[0] คือ key เดิม, round_key[1]..round_key[10] คำนวณต่อกันไปด้วย RotWord + SubWord + Rcon ตามหัวข้อ 7.1) — round_key[r] แต่ละชุดถูกใช้ใน Round r ตามลำดับด้านบน

แผนภาพ Data Path

แผนภาพด้านล่างแสดงเส้นทางข้อมูล (data path) แบบ iterative เดียวกับที่อธิบายไว้ข้างต้น ควบคุมโดย round_controller (FSM) — เห็นชัดว่า MixColumns ถูก bypass เฉพาะใน Round 10 เท่านั้น ส่วนโมดูลอื่นทำงานทุกรอบเหมือนกัน:



รูปที่ 1: Data path ของวงจร AES-128 Encryption (top-level) — Round 0 ทำ AddRoundKey อย่างเดียว, Round 1–9 ทำครบทั้ง 4 ขั้นตอน, Round 10 ข้าม MixColumns

5. ตัวอย่างการคำนวณจริงระดับระบบ (System-Level Worked Example)

ค่าด้านล่างเป็น official test vector ที่เผยแพร่โดย NIST ใน FIPS-197 — ตรวจสอบซ้ำแล้วด้วยไลบรารี pycryptodome (มาตรฐานอุตสาหกรรม) และด้วยการคำนวณ AES-128 จากศูนย์อีกชุดหนึ่ง (สร้าง S-box ด้วยสูตรคณิตศาสตร์ GF(2^8) inverse + affine transform ไม่ใช่ตารางที่พิมพ์มือ) ผลลัพธ์ตรงกันทั้งสองวิธี — ใช้ค่าเหล่านี้ตรวจสอบวงจรของนักศึกษาได้ทันที: ป้อน key และ plaintext ตามที่ระบุ วงจรต้องให้ ciphertext ตรงทุกบิต

Test Vector 1 — FIPS-197 Appendix B

```
key        = 2b7e151628aed2a6abf7158809cf4f3c
plaintext  = 3243f6a8885a308d313198a2e0370734
ciphertext = 3925841d02dc09fbdc118597196a0b32
```

Test Vector 2 — FIPS-197 Appendix C.1 (Known Answer Test)

```
key        = 000102030405060708090a0b0c0d0e0f
plaintext  = 00112233445566778899aabbccddeeff
ciphertext = 69c4e0d86a7b0430d8cdb78070b4c55a
```

6. สถาปัตยกรรม: การแตกเป็น Submodule (Top-Down Spec → Bottom-Up Build)

เขียนสเปคระดับบนสุด (หัวข้อ 2–4) ก่อน แล้วจึงแตกเป็น submodule ตามตารางนี้ — implement และ unit test ทีละโมดูลตามลำดับ bottom-up (คอลัมน์ขวาสุด) แล้วจึงรวมเป็น integration test สุดท้าย เหมือน pattern ที่เรียนในหัวข้อ 5 ของบทที่ 9:

#	โมดูล	หน้าที่	build/test order (bottom-up)
1	`key_expansion.v`	ขยาย cipher key 128 บิต เป็น 11 round key	1 (ก่อนสุด — ไม่พึ่งใคร)
2	`sub_bytes.v`	แทนที่แต่ละไบต์ด้วยค่าจากตาราง S-box	2
3	`shift_rows.v`	หมุนแต่ละแถวของ state ไปทางซ้ายตามจำนวนที่กำหนด	3
4	`mix_columns.v`	คูณเมทริกซ์ใน GF(2^8) ผสมแต่ละคอลัมน์เข้าด้วยกัน	4
5	`add_round_key.v`	XOR state กับ round key ของรอบนั้น	5
6	`round_controller.v`	FSM คุมลำดับ 11 รอบ, เลือกว่าจะ MixColumns หรือไม่, สร้าง busy/done	6
—	`aes_top.v`	ต่อทั้ง 6 โมดูลเข้าด้วยกัน	7 (integration, ลำดับสุดท้าย)

7. สเปค + พฤติกรรม + ตัวอย่างของแต่ละ Submodule

7.1 key_expansion.v

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
clk, rst_n	in	สัญญาณนาฬิกา/รีเซ็ต
key_in[127:0]	in	cipher key ตั้งต้น
key_valid	in	pulse เริ่มกระบวนการขยาย key
rk_index[3:0]	out	หมายเลข round key ที่กำลังส่งออก (0-10)
rk_data[127:0]	out	round key ของรอบนั้น
rk_valid	out	1 ไซเคิลต่อ round key ที่พร้อมใช้ (ควรจะพัลส์ 11 ครั้งต่อการ key_valid 1 ครั้ง)
rk_done	out	1 = ขยายครบ 11 round key แล้ว (round_key[0]..round_key[10] พร้อมใช้งาน)

พฤติกรรม (Behavior)

round_key[0] คือ key เดิม (key_in) ไม่ต้องคำนวณ

round_key[r] คำนวณจาก round_key[r-1] ทีละ 4 ไบต์ (word): word แรกของแต่ละ round key ใหม่ ได้จาก RotWord (หมุนไบต์ซ้าย 1 ตำแหน่ง) ตามด้วย SubWord (แทนที่แต่ละไบต์ด้วย S-box เดียวกับที่ใช้ใน SubBytes) แล้ว XOR กับค่าคงที่ Rcon[r] ที่ไบต์แรก แล้ว XOR กับ word แรกของ round_key[r-1] — ส่วนอีก 3 word ที่เหลือของ round_key[r] ได้จาก XOR สะสมของ word ก่อนหน้าใน round key เดิมและ word ก่อนหน้าใน round key ใหม่ (ตามสูตร Rijndael key schedule มาตรฐาน)

ตาราง Rcon อยู่ในภาคผนวก (หัวข้อ 10)

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ใช้ key = 2b7e151628aed2a6abf7158809cf4f3c (แบ่งเป็น 4 word: w0=2b7e1516, w1=28aed2a6, w2=abf71588, w3=09cf4f3c) คำนวณ w4 (word แรกของ round_key[1]):

```
w3            = 09cf4f3c
RotWord(w3)   = cf4f3c09   (หมุนไบต์ซ้าย 1 ตำแหน่ง)
SubWord(...)  = 8a84eb01   (แต่ละไบต์ผ่าน S-box อิสระ)
XOR Rcon[1]=0x01 กับไบต์แรก -> 8b84eb01
XOR กับ w0=2b7e1516 -> w4 = a0fafe17
```

w4 คือ word แรกของ round_key[1] จริง ๆ — ตรวจสอบได้จากตาราง round key ทั้ง 11 ชุดด้านล่าง (คำนวณจากศูนย์และตรวจสอบด้วย pycryptodome แล้วว่าถูกต้อง):

```
round_key[0]  = 2b7e151628aed2a6abf7158809cf4f3c
round_key[1]  = a0fafe1788542cb123a339392a6c7605
round_key[2]  = f2c295f27a96b9435935807a7359f67f
round_key[3]  = 3d80477d4716fe3e1e237e446d7a883b
round_key[4]  = ef44a541a8525b7fb671253bdb0bad00
round_key[5]  = d4d1c6f87c839d87caf2b8bc11f915bc
round_key[6]  = 6d88a37a110b3efddbf98641ca0093fd
```

```
round_key[7]  = 4e54f70e5f5fc9f384a64fb24ea6dc4f
round_key[8]  = ead27321b58dbad2312bf5607f8d292f
round_key[9]  = ac7766f319fadc2128d12941575c006e
round_key[10]  = d014f9a8c9ee2589e13f0cc8b6630ca6
```

7.2 sub_bytes.v

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
`data_in[127:0]`	in	state ขาเข้า (16 ไบต์)
`data_out[127:0]`	out	state ขาออก หลังแทนที่ทุกไบต์ด้วย S-box

พฤติกรรม (Behavior)

เป็น combinational logic ล้วน (ไม่มี clock) — แทนที่แต่ละไบต์ใน state ทั้ง 16 ไบต์ด้วยค่าที่สอดคล้องกันในตาราง S-box (256 entry lookup table) พร้อมกันในไซเคิลเดียว

S-box สร้างจากสูตรคณิตศาสตร์ตายตัว (GF(2^8) multiplicative inverse ตามด้วย affine transformation) แต่ในฮาร์ดแวร์มักนำไป implement เป็น ROM/case statement 256 entries ตรง ๆ เพื่อความเร็ว ไม่ต้องคำนวณ inverse แบบ real-time (ดูหมายเหตุในภาคผนวก)

ใช้ตารางเดียวกับที่ key_expansion.v ใช้ใน SubWord

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ป้อน state = 193de3bea0f4e22b9ac68d2ae9f84808 (ค่า state หลัง Round 0 AddRoundKey ของ Test Vector 1) เข้า sub_bytes ต้องได้ output ดังนี้ (คำนวณจริง ตรวจสอบแล้ว):

```
input  = 193de3bea0f4e22b9ac68d2ae9f84808
output = d42711aee0bf98f1b8b45de51e415230
```

ตัวอย่างระดับไบต์เดี่ยว (ง่ายต่อการตรวจ unit test ทีละไบต์): S-box[0x19] = 0xd4 (ไบต์แรกของ state), S-box[0x3d] = 0x27 (ไบต์ที่สอง)

7.3 shift_rows.v

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
`data_in[127:0]`	in	state ขาเข้า
`data_out[127:0]`	out	state ขาออก หลังหมุนแต่ละแถว

พฤติกรรม (Behavior)

เป็น combinational logic ล้วน (แค่การจัดเรียงสาย ไม่มีการคำนวณ) — มองไบต์ 16 ไบต์เป็นเมทริกซ์ 4x4 แบบ column-major (byte[r + 4c] คือแถว r คอลัมน์ c)

แถว 0: ไม่หมุน | แถว 1: หมุนซ้าย 1 ตำแหน่ง | แถว 2: หมุนซ้าย 2 ตำแหน่ง | แถว 3: หมุนซ้าย 3 ตำแหน่ง

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ตัวอย่าง (ใช้ state หลัง SubBytes ของ Round 1 ในหัวข้อ 7.2 เป็นอินพุต):

ก่อนหมุน (แถว0..แถว3, คอลัมน์0..3):

```
  d4 e0 b8 1e   (แถว 0 — ไม่หมุน)
  27 bf b4 41   (แถว 1 — จะหมุนซ้าย 1)
  11 98 5d 52   (แถว 2 — จะหมุนซ้าย 2)
  ae f1 e5 30   (แถว 3 — จะหมุนซ้าย 3)
```

หลังหมุน:

```
  d4 e0 b8 1e   (แถว 0 — เหมือนเดิม)
  bf b4 41 27   (แถว 1 — หมุนซ้ายแล้ว)
  5d 52 11 98   (แถว 2 — หมุนซ้ายแล้ว)
  30 ae f1 e5   (แถว 3 — หมุนซ้ายแล้ว)
```

รวมเป็น state 128 บิต: output = d4bf5d30e0b452aeb84111f11e2798e5

7.4 mix_columns.v

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
`data_in[127:0]`	in	state ขาเข้า
`bypass`	in	1 = ข้ามการคำนวณ (ส่ง data_in ผ่านตรงๆ) ใช้ตอน Round 10
`data_out[127:0]`	out	state ขาออก หลังผสมแต่ละคอลัมน์ (หรือเท่ากับ data_in ถ้า bypass=1)

พฤติกรรม (Behavior)

เป็น combinational logic — คำนวณแยกอิสระทีละคอลัมน์ (4 ไบต์) ด้วยการคูณเมทริกซ์คงที่ใน Galois Field GF(2^8) กับ irreducible polynomial ของ AES (x^8+x^4+x^3+x+1, เลขฐานสิบหก 0x11B)

สูตรต่อ 1 คอลัมน์ (a0,a1,a2,a3) -> (b0,b1,b2,b3): b0 = 2•a0 ⊕ 3•a1 ⊕ a2 ⊕ a3, b1 = a0 ⊕ 2•a1 ⊕ 3•a2 ⊕ a3, b2 = a0 ⊕ a1 ⊕ 2•a2 ⊕ 3•a3, b3 = 3•a0 ⊕ a1 ⊕ a2 ⊕ 2•a3 (เครื่องหมาย • คือการคูณใน GF(2^8), ⊕ คือ XOR)

ต้องมีทางเลือก "bypass" หรือกลไกเทียบเท่า เพราะ Round 10 ไม่ใช้ MixColumns เลย (ดูหัวข้อ 3 ข้อ 4) — round_controller.v เป็นตัวสั่งว่าจะ bypass หรือไม่ในแต่ละรอบ

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ตัวอย่างคอลัมน์แรกของ Round 1 (อินพุตมาจากผลลัพธ์ shift_rows ในหัวข้อ 7.3):

```
input column 0  = d4 bf 5d 30
b0 = 2•0xd4 ⊕ 3•0xbf ⊕ 0x5d ⊕ 0x30 = 0x04
output column 0 = 04 66 81 e5
```

คอลัมน์ทั้ง 4 รวมเป็น state 128 บิต: output = 046681e5e0cb199a48f8d37a2806264c

7.5 add_round_key.v

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
`data_in[127:0]`	in	state ขาเข้า
`round_key[127:0]`	in	round key ของรอบนั้น
`data_out[127:0]`	out	state ขาออก = data_in XOR round_key

พฤติกรรม (Behavior)

เป็น combinational logic ล้วน — XOR แบบบิตต่อบิตระหว่าง state 128 บิต กับ round key 128 บิต ของรอบนั้น ง่ายที่สุดในบรรดา 4 transform หลักของ AES

โมดูลนี้ถูกเรียกใช้ทุกรอบ (Round 0 ถึง Round 10) ต่างกันแค่ round_key ที่ป้อนเข้า

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ตัวอย่าง Round 1 (state หลัง MixColumns XOR กับ round_key[1]):

```
data_in    = 046681e5e0cb199a48f8d37a2806264c
round_key  = a0fafe1788542cb123a339392a6c7605
data_out   = a49c7ff2689f352b6b5bea43026a5049
```

ตรวจไบต์แรก: 0x04 XOR 0xa0 = 0xa4 — ตรงกับไบต์แรกของ data_out ด้านบน

7.6 round_controller.v (FSM)

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
`start, key_valid`	in	สัญญาณกระตุ้นจาก top-level
`round_num[3:0]`	out	รอบปัจจุบัน (0-10) — ป้อนให้ตัวเลือก round_key
`mc_bypass`	out	1 เมื่อ round_num=10 (สั่งให้ mix_columns.v ข้ามการคำนวณ)
`state_load`	out	สัญญาณควบคุม datapath ภายใน (โหลด plaintext, เก็บ state ระหว่างรอบ ฯลฯ — นักศึกษาออกแบบรายละเอียดเองได้)
`busy, done`	out	ส่งต่อไปเป็น output ของ aes_top.v โดยตรง

พฤติกรรม (Behavior)

เป็นหัวใจของการควบคุมลำดับ — ไม่ทำการคำนวณ AES เอง แต่กำหนดว่าจะใช้ round key ไหน (round_num), จะให้ mix_columns ทำงานหรือ bypass (เฉพาะ round 10 ที่ bypass), และเมื่อไหร่ที่ผลลัพธ์พร้อม (done)

state ของ FSM ที่แนะนำ (ปรับได้ตามการออกแบบจริง): IDLE → (key_valid) → KEY_EXPAND → (rk_done) → IDLE(busy=0, พร้อมรับ start) → (start) → ROUND0 → ROUND1..9 (full round) → ROUND10 (final round) → DONE(1 cycle) → กลับ IDLE

ข้อกำหนดตายตัวตามหัวข้อ 3-4: busy=1 ตลอดตั้งแต่เข้า KEY_EXPAND หรือ ROUND0 จนถึงก่อน DONE, mc_bypass ต้องเป็น 1 เฉพาะตอน round_num=10 เท่านั้น

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ตารางอ้างอิง transform ที่ต้องทำงานในแต่ละรอบ (ตรวจสอบแล้วจาก Test Vector 1 จริง — ใช้ตรวจ output ของแต่ละ state):

```
round_num = 0              -> AddRoundKey เท่านั้น (mc_bypass ไม่เกี่ยวเพราะไม่เข้า SubBytes/ShiftRows/MixColumns เลย)
round_num = 1..9           -> SubBytes -> ShiftRows -> MixColumns(mc_bypass=0) -> AddRoundKey
round_num = 10 (สุดท้าย)   -> SubBytes -> ShiftRows -> MixColumns(mc_bypass=1, ข้าม) -> AddRoundKey
```

ตัวอย่างผลลัพธ์ปลาย Round 10 ของ Test Vector 1: state เข้า round10 = eb40f21e592e38848ba113e71bc342d2, หลัง SubBytes = e9098972cb31075f3d327d94af2e2cb5, หลัง ShiftRows = e9317db5cb322c723d2e895faf090794, XOR round_key[10] (d014f9a8c9ee2589e13f0cc8b6630ca6) ได้ ciphertext สุดท้าย = 3925841d02dc09fbdc118597196a0b32 — ตรงกับค่า ciphertext ในหัวข้อ 5 ทุกบิต

7.7 aes_top.v (Integration — ลำดับสุดท้าย)

สเปค Input / Output

สัญญาณ	I/O	ความหมาย
(ตามหัวข้อ 2 ทั้งหมด)	in/out	aes_top.v มีพอร์ตเหมือน top-level interface ในหัวข้อ 2 ทุกประการ

พฤติกรรม (Behavior)

ต่อทั้ง 6 โมดูล (7.1–7.6) เข้าด้วยกัน: key_expansion เก็บ round key ทั้ง 11 ชุดไว้ใน register array, round_controller คุมว่าจะใช้ round key ชุดไหนและ mix_columns จะ bypass หรือไม่ในแต่ละไซเคิล, sub_bytes/shift_rows/mix_columns/add_round_key ต่อกันเป็นสายตามลำดับที่ round_controller กำหนด

เป็นลำดับ build/test สุดท้าย (bottom-up) — ต้อง unit test ทั้ง 6 โมดูลผ่านหมดก่อน จึงค่อยรวมเป็น integration test ตรวจด้วย 2 test vector ในหัวข้อ 5

ตัวอย่างการคำนวณ (Example — ตรวจสอบแล้วจริง)

ไม่มีตัวเลขคำนวณเพิ่มเติมสำหรับโมดูลนี้ — ผลลัพธ์ที่ต้องตรวจคือ ciphertext สุดท้ายของทั้งระบบ ต้องตรงกับหัวข้อ 5 ทุกบิต เมื่อป้อน key และ plaintext ตามที่ระบุ

8. เกณฑ์การยอมรับ (Acceptance Criteria)

12. Unit test ระดับ submodule: แต่ละโมดูลใน 7.1–7.6 ต้องผ่านตัวอย่างการคำนวณที่ให้ไว้ในหัวข้อนั้น ๆ อย่างน้อย 1 กรณี (แนะนำให้ทดสอบเพิ่มเติมด้วยอินพุตอื่นที่ AI/นักศึกษาสร้างเอง)
13. Integration test: ป้อน Test Vector 1 และ Test Vector 2 (หัวข้อ 5) เข้า aes_top.v ต้องได้ ciphertext ตรงทุกบิตทั้งสองกรณี
14. ต้องรัน simulation จริงด้วย Icarus Verilog (iverilog/vvp) หรือ ModelSim แล้วแนบ transcript ผลลัพธ์จริง — ห้ามใช้ตัวเลขที่ AI "อ้างว่า" ถูกต้องโดยไม่ได้รันจริง
15. พฤติกรรม busy/done/edge case ตามหัวข้อ 3 ต้องถูกทดสอบด้วย (เช่น กด start ซ้ำระหว่าง busy=1 ต้องไม่มีผลใด ๆ)

9. สิ่งที่ต้องส่ง (Deliverables)

- ไฟล์ RTL ทั้ง 7 ไฟล์: key_expansion.v, sub_bytes.v, shift_rows.v, mix_columns.v, add_round_key.v, round_controller.v, aes_top.v
- ไฟล์ testbench: unit test อย่างน้อยโมดูลละ 1 ไฟล์ (6 ไฟล์) + integration test 1 ไฟล์ (รวม 7 ไฟล์)
- Transcript ผลการรันจริงจาก iverilog/vvp (หรือ ModelSim) แสดงผล PASS/FAIL ของทุกเทส
- รายงานสั้น (ไม่เกิน 1–2 หน้า) สรุป: prompt หลักที่ใช้กับ AI, จุดที่ AI สร้างผิด/ต้องแก้เอง (ถ้ามี) พร้อมสาเหตุ, ผลลัพธ์สุดท้ายผ่านครบหรือไม่

10. ภาคผนวก

10.1 ตาราง Rcon (Round Constant)

ใช้ใน key_expansion.v — Rcon[i] คือไบต์แรกของ word คงที่ที่ XOR เข้ากับผลลัพธ์ของ SubWord(RotWord(...)) ทุกครั้งที่ i หารด้วย 4 ลงตัว (ตามหัวข้อ 7.1):

i	1	2	3	4	5	6	7	8	9	10
Rcon[i]	0x01	0x02	0x04	0x08	0x10	0x20	0x40	0x80	0x1B	0x36

10.2 หมายเหตุเรื่อง S-box และ GF(2^8)

S-box คือตารางลุคอัพขนาด 256 ค่า (1 ไบต์ต่อ 1 ไบต์) สร้างจากสองขั้นตอนทางคณิตศาสตร์ตายตัว: (1) หา multiplicative inverse ของไบต์นั้นใน Galois Field GF(2^8) ที่นิยามด้วย irreducible polynomial x^8+x^4+x^3+x+1 (0x11B) — โดยกำหนดว่า inverse ของ 0 คือ 0 — แล้ว (2) นำผลลัพธ์ผ่าน affine transformation คงที่ (b' = b ⊕ rotl(b,1) ⊕ rotl(b,2) ⊕ rotl(b,3) ⊕ rotl(b,4) ⊕ 0x63) ในฮาร์ดแวร์จริงมักไม่คำนวณสองขั้นตอนนี้แบบ real-time แต่ implement เป็น ROM หรือ case statement 256 entries ตรง ๆ เพื่อความเร็วและความง่าย — S-box เดียวกันนี้ถูกใช้ทั้งใน sub_bytes.v และใน SubWord ของ key_expansion.v

10.3 เอกสารอ้างอิง

NIST FIPS-197, "Advanced Encryption Standard (AES)" — มาตรฐานฉบับทางการที่นิยามอัลกอริทึม AES-128/192/256 ทั้งหมด รวมถึง test vector ที่ใช้ในเอกสารนี้ (Appendix A: Key Expansion Example, Appendix B: Cipher Example, Appendix C.1: AES-128 Known Answer Test)